DataHorizon implementation plan

Draft 0.2 | October 31, 2026

The first wireframes establish the visual direction for the desktop application. This plan turns them into a sequence of implementation increments with explicit dependencies and completion criteria. It is planning documentation, not a start of application development or a release-date commitment.

The **PDF edition** accompanies the **18-page wireframe set**. **Wireframe notes** record the interaction proposals. Story IDs refer to the **20 everyday spreadsheet stories**. The architectural contracts in **AGENTS.md** and **architecture.md** continue to apply.

1. Delivery goal and scope

Deliver a native Qt Widgets/C++ spreadsheet experience on Windows, macOS, and Linux. A user must be able to complete the Supplies session without writing SQL or Python: create a workbook, enter data, calculate, format, filter, correct, reopen, and export a copy. Storage, calculation, and operation outcomes must be visible and trustworthy.

The first desktop preview covers the 20 everyday stories within declared formula, format, interchange, and undo compatibility tiers. Unsupported features return specific errors or require explicit conversion choices. A smaller intermediate preview must identify its missing story criteria rather than claiming complete coverage.

Normal GUI operations use REST and the shared engine. CLI, SDK, and integrations use the same managed behavior. Independent SQLite tools remain a separate access path without Python edit handlers or application recalculation. Refresh does not replay skipped handlers. Business workflows remain in user code.

Full relational tooling, SQL authoring, Python authoring, and remote deployment follow the everyday desktop foundation. Their architectural needs are included early, especially change capture, identity, and service-owned Python execution. Concurrent multiuser editing and complete Excel parity are not preview promises.

2. Sequence and dependencies

Increment	Deliverable	Depends on	Wireframe pages
M0	Interaction and data contracts	Existing design	13, 17, 18
M1	Evidence for foundational choices	M0 specifications	2, 4, 6, 14
M2	One complete managed edit	M1 decisions	2-4, 13-14
M3	Formula-driven Supplies sheet	M2	4-6
M4	Everyday range and workbook editing	M3	5, 7-8, 11
M5	Sort, filter, find, and CSV	M4	9-12, 15
M6	Native lifecycle and spreadsheet interchange	M5	3, 12-15
M7	Cross-platform desktop preview	M6	1-18
M8	Relational, programming, and remote expansion	Relevant M1-M7 contracts	Later wireframes

M0 is the next documentation task. Executable experiments, scaffolding, and application code begin only after a separate instruction to start implementation. Within each later increment, deliver the engine operation and its UI together; avoid accumulating screens that cannot perform their advertised actions.

3. M0 - Resolve the interaction contracts

Record decisions in the existing design documents and update affected wireframe annotations. The visual review does not settle the proposals below.

Decision	Required outcome	Needed before
Persistence and Save	Choose automatic per-operation storage or another explicit model; define cell drafts, first Save, Save a copy, close, and recovery	M2 lifecycle
Operation outcomes	Define rejected, rolled back, committed, calculation pending, follow-up failed, and outcome unknown; define lookup and retry	M2 mutations
Values and references	Define blank/NULL/text, numbers, date/locale rules, relative/absolute references, insertion, deletion, copy/cut, sort, and duplication	M3-M5 operations
Undo and history	Name supported actions, grouping, lifetime, handler behavior, redo invalidation, and external-change gaps	M4 undo
List and record scope	Define a declared freeform list, header handling, stable targets, filter edits, and hidden/filtered paste behavior	M5 lists
Compatibility tiers	Publish initial formula/function, style, and file-feature support separately for import and export	M3 and M6
Jobs and limits	Define atomic versus chunked commits, cancellation boundaries, progress, viewport limits, and benchmark budgets	M1 measurements; later jobs
Code activation	Define service-side activation, required-handler failure, controlled test fixtures, and external-side-effect semantics	M2 handler execution

Recommended starting proposal: retain the wireframe's automatic storage per submitted managed operation. An editor draft is cancellable; an accepted operation commits with its transactional hooks; Save finishes pending work and confirms the known result. Already committed edits require Undo to reverse. This remains a proposal until documented as a decision.

An untitled workbook also needs a defined temporary-storage and recovery policy. Decide what Saved means before a permanent location exists. A lost response must not be treated as proof of rollback or trigger a blind retry of workbook code.

Complete when: each decision has an explicit initial contract, or a narrow unsupported case and the milestone that must resolve it. Supply keyboard/focus flows, loading/empty/read-only states, and a numeric benchmark specification.

4. M1 - Prove the foundational choices

After implementation is requested, use bounded experiments to select technology and confirm behavior. Record alternatives, measured results, and limitations. Experiments are evidence for decisions, not production feature completion.

- Compare the SQLite binding and capture options for actual managed changes. Exercise inserts, updates, deletes, upserts/replacements, triggers, cascades, failures, rollback, and unsupported statement classes. Verify old/new values and bounded change iteration. Direct access must not require Python callbacks.
- Compare service framework and Python execution arrangements. Prove that handler-generated edits join their parent transaction, handler failure rolls back the whole change, and cancellation has a known outcome on all platforms. Keep engine calls inside a transaction-aware context rather than loopback REST.
- Validate the physical representation of sparse sheets, inputs versus derived results, identities, styles, and format versions. Test ordinary external SQLite inserts and conservative refresh/reopen behavior.
- Exercise a Qt table-view candidate with service-backed viewports, scrolling, distant address jumps, range selection, edit delegates, clipboard, and frozen panes. Measure cache size, request count, input responsiveness, and cell geometry.
- Choose the supported Qt/toolchain baseline, build and packaging approach, dependency versions, and test tooling. Qt Widgets/C++ and GUI GPLv3 are already selected; service language, other-component licenses, and remaining technology choices remain open. The external client SDK is Python.

Complete when: the selected approaches satisfy the M0 budgets and correctness cases, or unsupported capabilities are explicitly restricted. Record the managed SQL support matrix before exposing any SQL mutation. Resolve grid address/extent limits instead of promising unlimited rows from theoretical SQLite capacity.

5. Implementation responsibilities

Use these boundaries to keep client behavior consistent. Names below describe responsibilities rather than prescribing source directories or class names.

Area	Responsibility	Boundary
Desktop shell	Menus, toolbars, sheet tabs, dialogs, docks, keyboard focus	No direct workbook database writes
Grid model and delegates	Viewport cache, selection, source editor, pending/error display	No whole-workbook cache or cell-by-cell REST loop for bulk edits
Desktop REST client	Typed requests, operation IDs, expected state, bounded reads, status reconciliation	No automatic retry of an unknown mutation outcome
Service and engine	Transactions, rules, ordering, calculations, jobs, history, SQL policies	Same operations for all managed clients
Storage and format	SQLite inputs/metadata, versions, snapshots, recovery	Independent SQLite access stays possible
Interchange	Staging, conversions, supported translations, snapshot exports	No silent coercion, truncation, or unsupported-code activation

Map the shell to QMainWindow, shared QActions, QToolBar, QTabBar, QDockWidget, and standard dialogs as described in the wireframes. Start with a QTableView-based candidate, with custom spreadsheet behavior contingent on M1 results. Keep all Qt view/model mutations on the GUI thread; service work must not block input. QNetworkAccessManager is a candidate for asynchronous REST transport. Notification transport remains open.

Treat editing, storage, calculation, and follow-up work as separate state dimensions. For example, Saved plus Calculating is valid; a follow-up error cannot convert committed data into an unsaved draft. Associate replies with the workbook, sheet, request, and input state so stale responses cannot repaint the wrong view.

6. M2 - Deliver one complete managed edit

Goal: make the smallest useful GUI-to-service-to-SQLite path real.

- Establish the build, automated checks, and smoke packaging on all three systems. Build the main shell, a blank sheet, source/address input, sheet tab, and status.
- Start or attach to the authenticated localhost service. Distinguish a desktop-owned service from an independently running service during shutdown. Define connection loss, incompatible service versions, and startup failure.

- Implement native create/open, read-only detection, bounded range reads, typed single-cell mutation, and operation-status lookup. Publish OpenAPI and an interactive API explorer with the first implemented resources.
- Make Enter submit, Escape cancel the editor draft, and accepted/rejected edits reconcile with the grid. Implement the chosen first-save and close contract.
- Add a controlled Python handler fixture that validates a value and changes a second cell. Prove the same behavior through GUI, REST, a minimal managed CLI, and a thin Python client. Show a required handler that fails to load as an error.
- Include conservative external-edit detection and stale-write rejection before ordinary edits can silently overwrite independently changed data. An unsupported live schema change must produce an actionable stop or reopen requirement.

Complete when: a valid edit and its handler-generated edit commit together; a rejected edit changes neither cell; the file reopens with the stored input. A lost response is reconciled without repeating hooks. A direct SQLite edit runs no handler; refresh/reopen exposes it and invalidates derived state without replaying historical events. The service works without a desktop session.

7. M3 - Calculate the Supplies sheet

Goal: complete entry, formula editing, and calculation using the sample data.

- Implement typed entry and Text formatting for identifiers such as 00123. Keep raw input, formula source, displayed text, and cached results distinct.
- Add cell/range navigation, header selection, source editing, and explicit unsupported-address errors. Scrolling and selection must not emit edit events.
- Implement the declared first formula tier: arithmetic, cell/range references, absolute reference components, cross-sheet references, SUM, and IF. Publish supported operators, coercions, numeric behavior, and function signatures. Exercise cross-sheet behavior with fixtures before M4 adds full sheet management.
- Add dependency invalidation, input-state-associated results, pending cells, circular-reference detection, and recoverable formula errors in the Issues dock.
- Implement formula fill-down as one managed operation for the sample session. Enforce the M0 rule for calculations required by transactional validation.

Complete when: D2:D4 evaluate to 7.00, 5.00, and 6.00; D6 totals 18.00. Changing Pen quantity from 4 to 6 produces 7.50 and total 20.50. An older calculation cannot overwrite the newer result. Fixing SUUM to SUM recovers the formula without re-entering its source. Reopen never labels unverified results as current.

8. M4 - Complete everyday range and workbook editing

Goal: support familiar editing and presentation without compromising atomicity.

- Implement copy/cut/paste, tab-separated clipboard input, values-only and formats-only paste, constant/formula fill, and explicit numeric-series fill. Use one bulk engine command for each ordinary range operation.
- Implement named Undo/Redo for the declared supported edits. Capture enough engine-side before/after state for original and handler-generated changes. Define reversal as a new managed operation, including rule rejection.
- Add sheet create/rename/reorder/duplicate/delete, row/column insert/delete, reference adjustment, and broken-reference warnings. Keep stable identity separate from displayed position and names.
- Add supported number formats, emphasis, alignment, borders, fills, wrapping, row/column sizing, freeze, hide/unhide markers, and saved view state. Fit-column measurement must stay within the declared sampling/read budget.
- Restore platform conventions for menus, shortcuts, dialog buttons, focus, high-DPI geometry, toolbar overflow, and dock visibility.

Complete when: one rejected paste or fill leaves all target cells unchanged; cut never loses its source after a rejected destination write. One Undo reverses a supported paste including applicable generated edits, or reports rejection without partial reversal. Structure changes follow the reference contract. Formatting changes display while retaining inputs and formulas across reopen.

9. M5 - Add list operations, search, and CSV

Goal: organize and exchange ordinary datasets using bounded service operations.

- Implement declared list scope, explicit header handling, multikey sort, and value/comparison filters. Execute full-data sort/filter in the engine, not only against the desktop's loaded rows. Preserve complete records and stable targets.
- Show active criteria, matching/total counts, no-match states, and Clear filters. Explain when an accepted edit makes its row disappear from the filtered view.
- Implement scoped find/replace, displayed-value versus formula-source search, locations, match counts, and Replace All review. A rejected ordinary Replace All rolls back as one operation.
- Add CSV preview, encoding/delimiter controls, explicit column typing, ambiguous date/conversion reports, and atomic import. Add consistent-snapshot values export.
- Add progress and cancellation for long operations. Enforce ordinary-operation size limits before mutation; offer an explicitly reviewed large-job path. Chunked iteration must not silently become chunked commits.

Complete when: Quantity > 2 shows Pen and Folder, and clearing the filter restores Notebook. Sorting keeps labels, quantities, prices, and identities paired. Find/replace respects scope. CSV retains text IDs and exports total 20.50. Cancellation reports either confirmed rollback or the declared committed batches; it never implies that committed work was discarded.

10. M6 - Finish file lifecycle and interchange

Goal: complete the import/export and recovery screens within a tested tier.

- Complete temporary-workbook recovery, first Save, Save a copy, read-only files, close during pending work, permission/disk failures, and service restart. Use consistent database snapshots for native copies and backups.
- Implement Excel and OpenDocument import/export in declared stages. Track values, formulas, formatting, names, tables, and advanced features separately; import and export support do not imply each other.
- Show actual affected features and locations before lossy conversion. Require an explicit supported fallback for unsupported formulas. Imported code cannot activate itself. Preserve native code, queries, and relationships in the source.
- Export from a consistent stored state with current calculations, or block with actionable pending/error details. Report target size limits without truncation.
- Complete rollback versus committed/follow-up-failed reporting. If follow-up execution is included, define external-side-effect, retry, and recovery semantics before enabling it; do not claim exactly-once external effects.

Complete when: the complete Supplies walkthrough passes with reopen and declared Excel/ODS/CSV fixtures. Conversion reports match actual omissions and transformations. Failure leaves the native source intact and identifies any partial destination. Restart preserves committed inputs and pending calculation state without inventing successful follow-up work.

11. M7 - Qualify the cross-platform desktop preview

Goal: make the implemented everyday scope reviewable on supported systems.

- Run the story acceptance matrix on Windows, macOS, and Linux. Include keyboard workflows, screen-reader naming/navigation, high DPI, system palette, narrower windows, native dialogs, clipboard interchange, and dock/view restoration.
- Exercise connection loss, busy databases, concurrent independent writers, stale edits, handler exceptions/timeouts, calculation races, unknown commit outcomes, process termination, file corruption detection, and interrupted jobs.
- Verify native format checks/migrations, consistent backup/restore, crash recovery, local credentials, and bundled Python/runtime behavior.

- Package installers/distributions and test from a clean machine. Include GUI GPLv3 materials and record the selected licenses and distribution obligations for other components and dependencies before distributing them.
- Publish measured limits and compatibility matrices. Make unavailable features explicit instead of retaining enabled controls that silently do nothing.

Complete when: every claimed story criterion has passing evidence on the supported platform matrix, the Supplies session passes end to end, measured budgets are met, and unresolved issues have explicit user-visible limits. Release notes distinguish known limitations from completed behavior.

12. Verification and performance plan

Keep tests centered on contracts rather than the appearance of implementation. Use shared fixtures across access paths and compare stored inputs, operation outcomes, handler effects, and freshness, not just visible success messages.

Test layer	Required evidence
Engine and native format	Atomic mutations, reference transforms, formula errors, migration, backup/recovery
Managed-path integration	Equivalent GUI/REST/CLI/SDK results and rules; SQL parity as its supported surface expands
Qt model and interaction	Model consistency, editing/focus, selection, async reply ordering, keyboard and clipboard
Interchange	Round-trip fixtures per feature tier; accurate conversion reports and overflow errors
Failure and concurrency	Rollback, lost acknowledgement, stale writes, direct edits, cancellation, follow-up failure
Cross-platform acceptance	All 20 claimed story contracts and the Supplies session on packaged builds

Use Qt Test for interaction checks and QAbstractItemModelTester for model invariants. Add separate destructive-operation tests; model consistency alone cannot prove correct paste, sort, or rollback behavior.

During M0 specify reference hardware, dataset shapes, maximum live cache bytes, viewport request sizes, and p95 targets for first usable viewport, distant jump, scrolling, edit acknowledgement, and recalculation. In M1 measure sparse and dense sheets, long text/wide columns, formula chains, indexed lists, and a disk dataset larger than the declared memory budget. Record cold/warm runs, total rows/cells, peak GUI/service memory, request counts, cancellation latency, and disk footprint. Fill in numeric targets before the experiment is accepted; do not defer all performance work until release or claim unmeasured unlimited scale.

13. M8 - Expand the product after the everyday foundation

Keep these as separate, dependency-driven increments with additional wireframes:

- **Relational work:** define sheet/table/view mapping and range promotion, typed records and keys, relationships, joins, read-only saved views, indexes, and SQL tools. Managed SQL mutations ship only for the M1-proven capture surface, with coherent transaction, DDL, metadata, and reference policies.
- **Programming work:** workbook modules/editor, explicit macros, custom Python functions, package environments, SDK completion, durable follow-up work where proven, and reproducible service-side execution. Refine recursion, ordering, dependency, trust, cancellation, and external-side-effect contracts.
- **Remote work:** opt-in service binding with TLS, mTLS, username/password authentication, actor identity, permissions, upload/download semantics, and interactive OpenAPI documentation. Keep remote paths distinct from local paths. Establish the supported concurrent-client model before promising collaboration.

The supporting native metadata and engine boundaries must accommodate these features earlier; their full user interfaces do not block the everyday preview.

14. First work package and completion record

The next work package is M0: settle persistence/outcome semantics, draft the value/reference and undo contracts, specify the first compatibility tiers, and refine missing state/focus wireframes. Produce the M1 experiment specifications and numeric performance budget before requesting implementation work.

For each later increment, record scope and exclusions, affected story criteria, decision/document changes, automated and manual evidence, platform results, remaining risks, and the next dependency. Estimate calendar effort after M1, when toolchain, storage, Python integration, and grid risks have measured results.

This plan creates no application code, project scaffolding, or executable UI prototype. The next implementation step remains a separate request.